Disaster Communication and Resource Management System

How it works, what it is used for, and why each part was chosen

1. The problem it solves

During floods, fires and similar emergencies, the people who must respond usually learn about the event **late and second-hand**: a phone call, a message, a neighbour. Control rooms then have to decide which resources to send with incomplete information. Worse, the very thing that makes an emergency dangerous, such as heavy rain, flooding or a fire, often **knocks out the Wi-Fi, mobile network and Internet** that a monitoring system depends on.

This project builds a small system that:

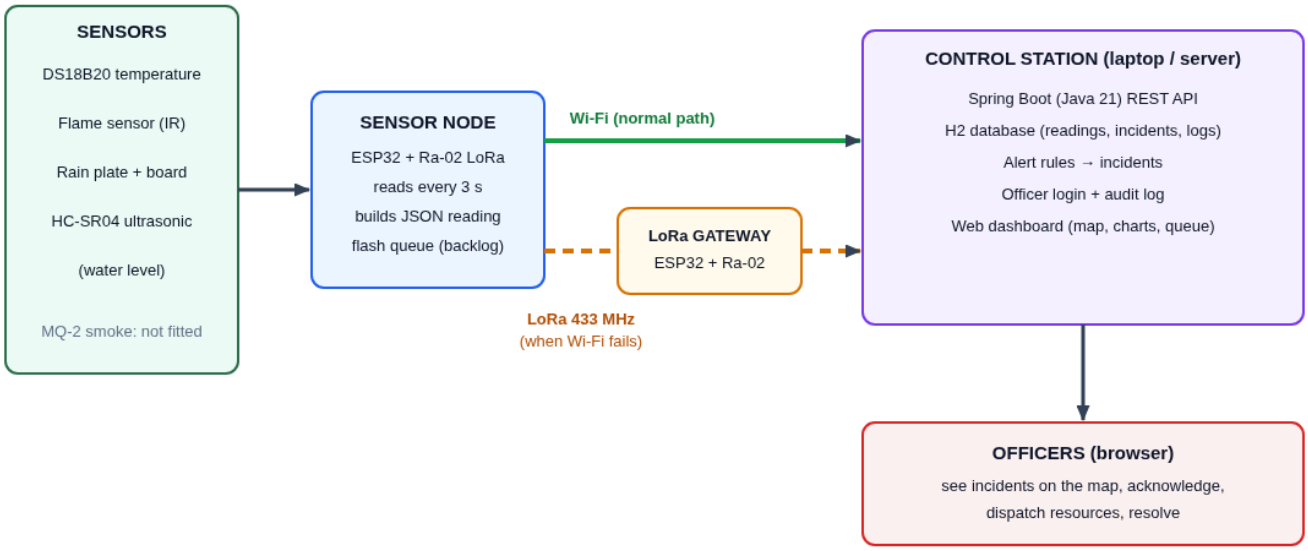
1. **Measures** the danger directly with sensors placed in the field (water level, rain, flame, temperature).
2. **Reports it automatically** over Wi-Fi, and if Wi-Fi or the Internet is down, switches to **LoRa radio**, which works over long distances with no network at all.
3. **Gives the control room one screen**: where each sensor is, what it reads, which incidents are open, and which rescue resources are free.
4. **Records the response**: an officer acknowledges the incident, dispatches the suggested resources, and resolves it, and the system logs who did what and when.

2. Practical applications

Where	What the system does there
Flood-prone streets, bridges, culverts, drains	The ultrasonic sensor hangs over the water and measures its level. Rain and water level together give an early warning before roads are cut off.
Warehouses, hostels, labs, godowns	The flame sensor and temperature probe detect fire or overheating, and the control room is alerted without anyone being present.
Villages and remote areas with weak or no Internet	LoRa carries small sensor messages over distances Wi-Fi cannot, to a gateway that forwards them.
Relief camps and temporary shelters	Quick to set up with cheap parts, and the same dashboard tracks incidents and available resources.
College campuses and industrial sites	A local control station on the campus network keeps working even when the Internet connection is lost.

Value over a phone call: detection is automatic and timestamped, the same screen shows which resources can help, and every action is logged for accountability and later review.

3. System overview



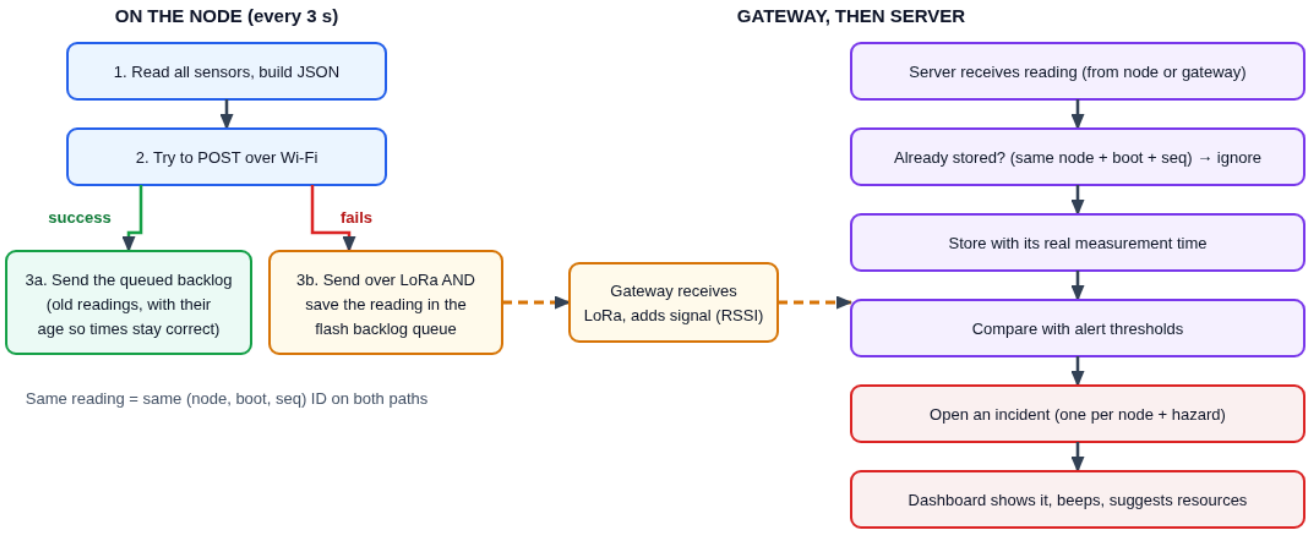
There are **three kinds of devices**:

- **Sensor node:** an ESP32 microcontroller with the sensors attached and a LoRa radio. It measures every 3 seconds and sends the reading.
- **Gateway:** a second ESP32 with a LoRa radio. It listens for radio messages and forwards them to the server over Wi-Fi.
- **Control station:** a laptop (or any computer) running the server and the dashboard that officers use in a browser.

4. How the whole system works, step by step

1. **Sense.** Every 3 seconds the node reads the temperature probe, flame sensor, rain plate and ultrasonic sensor. It packs the values into a short JSON message with a unique ID made of the node name, a random boot ID and a counter (`node1`, boot, sequence number).
2. **Try Wi-Fi first.** The node sends the message to the server over Wi-Fi using an HTTP POST. This is the normal path.
3. **If Wi-Fi or the server is unreachable**, the node does two things at once: it **transmits the same message over LoRa**, and it **saves it in its flash memory queue** (up to 200 messages).
4. **Gateway relay.** The gateway receives the radio message, adds the **signal strength (RSSI)** and forwards it to the server over its own connection.
5. **Catch-up.** When the node can reach the server again, it first sends the new reading, then replays the saved queue. Each replayed message carries its **age**, so the server records the time it was *measured*, not the time it finally arrived.
6. **No duplicates.** The same reading may arrive twice (once by LoRa, once by replay). The server recognises the repeated (node, boot, sequence) ID and **stores it only once**.

7. **Decide.** The server compares each reading with the **alert thresholds** (for example flood at 50 cm of water, flame detected, temperature 50 °C, rain 80 %). A reading over a limit opens an **incident** (one open incident per node and hazard type, so a sensor that stays in alarm does not spam).
8. **Respond.** The dashboard shows the incident at the top with a severity label, plays an alert sound if enabled, and **suggests the resources** that fit the hazard (for example a rescue boat and relief kits for a flood). An officer signs in, acknowledges, dispatches resources, and resolves the incident. Each action is written to an activity log with the officer's name.



5. The hardware, component by component

5.1 ESP32 development board (the brain of the node and gateway)

What it is: a low-cost microcontroller with a dual-core processor, built-in Wi-Fi and Bluetooth, many input/output pins, analog-to-digital converters and flash memory. **How it works here:** it runs our firmware in a loop: read the sensors, build the message, send it, wait. It also stores the offline queue in its flash memory. **Why this one:** it has Wi-Fi built in (no extra module), enough pins for four sensors plus the radio, and is cheap and widely supported. Two boards are used, one as the node and one as the gateway.

5.2 Ra-02 LoRa radio module (SX1278 chip, 433 MHz)

What it is: a radio chip that uses **LoRa** modulation, which spreads the signal over a wider band so it can be received at very low power levels. **How it works here:** the ESP32 talks to it over the **SPI** bus (wires: NSS, SCK, MISO, MOSI plus power and ground). The node transmits the JSON reading as a packet, and the gateway's identical module receives it. Each radio needs its antenna fitted before transmitting. **Why this one:** LoRa trades speed for range and low power, which suits tiny sensor messages. Typical LoRa links reach kilometres in open ground, though **we have only tested it at bench distance** (signal about −46 to −52 dBm). **Regulation note:** these modules are 433 MHz. India's licence-exempt band for LoRa is **865–867 MHz**, so a real deployment should use a module for the legally permitted band. 433 MHz is acceptable for a low-power classroom prototype.

5.3 DS18B20 temperature probe

What it is: a digital temperature sensor in a waterproof stainless probe, accurate to about ±0.5 °C. **How it works:** it sends the temperature as a digital number over a single data wire (the **1-Wire** protocol). It needs a **4.7 kΩ pull-up resistor** between its data and power lines so the data line returns to a known "high" level. **Why this one:** digital (no calibration or analog noise), waterproof, and it needs only one pin. It detects overheating and supports fire detection.

5.4 Flame sensor (HW-072)

What it is: a small board with an infrared photodiode and a comparator chip (LM393). **How it works:** flames emit infrared light. The photodiode sees it, the comparator compares it with a level set by the blue adjustable knob, and the board outputs a simple **digital signal**: LOW when a flame is present. **Why this one:** cheap and fast, and gives a clear yes/no, which is what fire detection needs. It senses a flame at short range and is not a replacement for a proper fire-detection system.

5.5 Rain sensor (plate and control board)

What it is: a printed plate of interleaved copper traces plus a small comparator board. **How it works:** dry, the traces are not connected. Water drops bridge them and change the resistance. The board turns this into an **analog voltage** (AO pin), which the ESP32's analog-to-digital converter reads. Firmware converts the reading to a 0–100 % wetness scale. **Why this one:** very cheap and gives a graded value (not just wet/dry). We power it from a GPIO pin so it can be switched off to reduce corrosion of the plate.

5.6 HC-SR04 ultrasonic distance sensor (water level)

What it is: the sensor that looks like two small "eyes": one is a speaker, the other a microphone, working at 40 kHz. **How it works:** the ESP32 sends a short trigger pulse. The sensor sends a burst of ultrasound, which bounces off the surface (the water). The **echo pin stays high for the round-trip time**. Distance in cm is the echo time in microseconds divided by 58. The sensor is mounted at a known height above the dry bed, so:

> **water level = mounting height − measured distance** (our setting: 70 cm)

Why this one: it measures without touching the water, so it does not corrode or get fouled, and it is very cheap. It runs at 5 V and its echo output is 5 V, so a **voltage divider (2.2 kΩ and 4.7 kΩ)** drops it to about 3.4 V, safe for the ESP32.

5.7 MQ-2 smoke sensor (not fitted)

Planned for smoke detection. It is **not installed** in the current build. The firmware reports "no sensor" for smoke, and the dashboard shows it as not available instead of showing noise.

5.8 Supporting parts

- **Resistors:** the 4.7 kΩ pull-up for the temperature probe, and the 2.2 kΩ / 4.7 kΩ voltage divider for the sonar echo line.

- **Breadboards and jumper wires:** build and change the circuit without soldering. The radio modules are connected with soldered wires, because breadboard contact was unreliable for them.
- **USB cables and a power bank:** the node can run from a power bank, so it is standalone.
- **Antennas:** one for each radio. Never transmit without one.

5.9 Pin map (node)

Function	ESP32 pin
LoRa radio: NSS, SCK, MISO, MOSI	D5, D18, D19, D23
Temperature probe: data / power	D4 / D15
Flame sensor DO	D27
Rain sensor AO / power	D32 / D13
Sonar Trig / Echo (through divider)	D25 / D33
Smoke sensor (not fitted)	D34

Analog sensors use pins on the ESP32's ADC1, because the ADC2 pins cannot be used while Wi-Fi is on.

6. The software, component by component

6.1 Firmware (runs on the ESP32s)

Language and tools: C++ with the Arduino framework, built and uploaded with **PlatformIO**. **Why:** the Arduino framework has ready libraries for every part we use, and PlatformIO builds the project the same way on Linux and Windows. **Libraries:** *LoRa* (radio), *OneWire* and *DallasTemperature* (temperature probe), *LittleFS* (the offline queue in flash), *HTTPClient* (sending to the server). **Key behaviours:** Wi-Fi first with LoRa fallback; flash backlog with replay; a unique (boot, sequence) ID on every reading; the node's uptime and each message's age, so the server can keep true measurement times; protection so the node never hangs if the radio is missing.

6.2 Server (Spring Boot, Java 21)

What it is: a web application exposing a REST API and serving the dashboard. **Why Java and Spring Boot:** it is a standard, widely used stack for building reliable server software and was the stack required for this project. Spring provides the web layer, database access, validation and testing tools. **Main pieces**

- **Database (H2, stored in a file):** readings, incidents, resources, node names and positions, accounts, settings and the activity log. **Why H2:** it is embedded, so there is nothing separate to install, which also makes it easy to run on any laptop.

- **Ingest and deduplication:** accepts readings from nodes and the gateway, ignores repeats of the same (node, boot, sequence).
- **Alert rules and incidents:** configurable thresholds; one open incident per node and hazard type.
- **Suggested resources:** a rule table: flood to rescue boat and relief kits; fire to fire engine and ambulance; smoke to fire engine; high temperature to ambulance.
- **Officer login and audit log:** passwords are stored as salted **PBKDF2** hashes, never as plain text. Viewing is open, and every change needs a login. Every action is recorded with the officer's name.
- **Tests:** five automated tests cover ingest, duplicates, the officer workflow, login rules, thresholds and replay timing.

6.3 Dashboard (web page)

Built with: plain HTML, CSS and JavaScript (no framework), **Leaflet** with OpenStreetMap for the map, and charts drawn on a canvas. **Why:** no build step, loads fast on any browser, and nothing to install.
What it shows

- **Field map:** nodes as markers coloured by state; a "My location" button using the browser's location service (accuracy depends on the device, laptops are often rough).
- **Incident queue:** sorted by severity (critical or warning), with the observed time, the reading against its threshold, and action buttons.
- **Sensor network table and charts:** live values, link type (Wi-Fi or LoRa) and signal strength, and 30-minute history with the threshold drawn on it.
- **Resources and activity log:** who is free, who is dispatched, and who did what.
- **Alerts:** optional sound and browser notification when a new incident opens or a node goes offline.
- **Thresholds dialog:** officers can change the alert limits without restarting.

7. Why these design choices

Choice	Reason
Local control station, not a cloud service	It keeps working when the Internet is down, which is exactly when it is needed.
LoRa as the backup link	Long range and low power, and independent of Wi-Fi and mobile networks.
Wi-Fi as the primary link	Faster and carries more data when it is available.
Flash queue and replay	No data is lost during an outage, and nothing is duplicated afterwards.
Unique (node, boot, sequence) IDs	The server can tell a repeated reading from a new one, whichever path it took.
Real measurement time on replayed data	Charts and incident times stay truthful after an outage.
Thresholds editable on the dashboard	Real sites need different limits, and the demo can trigger alerts on demand.
Officer login and audit log	Accountability: every dispatch decision is traceable.
Cheap, common parts	Easy to replicate and replace.

8. What was tested

Test	Result
Sensors to dashboard over Wi-Fi	Working, with a new reading every ~3 seconds
Node's Wi-Fi path cut while the gateway stays connected	The node switched to **LoRa**; 32 consecutive readings arrived through the gateway at about −52 dBm; **none missing**
Wi-Fi path restored	The node returned to Wi-Fi by itself; **no duplicates**, no gaps
Server taken offline for 40 seconds	All 31 readings made during it were stored after recovery, **none missing, none duplicated**, with correct times
Automated server tests	5 of 5 pass

9. Limits and honest notes

- **LoRa range is untested.** Only bench distance has been measured. An outdoor range test is still to do before quoting a figure.
- **Smoke sensing is not fitted**, so fire detection uses the flame sensor and temperature.

- **Frequency:** the 433 MHz modules are fine for a prototype, but a real deployment in India should use the licence-exempt 865–867 MHz band.
- **It is a breadboard prototype.** A field unit needs a proper board, a weatherproof case and a battery or solar supply.
- **Thresholds and the sonar mounting height are bench values** and need calibration at a real site.
- **No SMS or phone alerts yet.** Notifications are on the dashboard only.
- **Security is LAN-level.** There is a login and audit trail, but the default admin password must be changed, and the system is not designed to be exposed to the open Internet.
- **The map needs Internet** for map tiles. Everything else works offline.

10. Possible next steps

SMS or Telegram alerts to officers; the MQ-2 smoke sensor; solar power and battery monitoring; more nodes and a gateway at a high point; an outdoor range test; moving to the 865–867 MHz band; and a printed circuit board in a sealed enclosure.

Glossary

Term	Meaning
ESP32	A microcontroller with Wi-Fi. The "brain" of the node and gateway.
LoRa	A long-range, low-power radio technique for small messages.
RSSI	Received signal strength, in dBm. Closer to 0 is stronger (−50 is strong, −120 is very weak).
GPIO	A general-purpose pin on the ESP32 that can read or drive a signal.
ADC	Analog-to-digital converter: turns a voltage into a number.
Pull-up resistor	A resistor holding a signal line at "high" until something pulls it low.
Voltage divider	Two resistors that scale a voltage down, here 5 V to about 3.4 V.
REST API / JSON	The way programs exchange data over HTTP, using a simple text format.
Incident	A hazard alert that officers acknowledge, act on and resolve.